

10 状态模式

事物的状态



我原本以为在对象村一切都很容易，但现在每次我一回头就有变更请求进来。我快崩溃了！噢，也许我应该一直去参加Betty周三晚上的模式小组会。看我现在的状态！

鲜为人知的事实：策略和状态模式是双胞胎，刚出生就分开了。

你以为他们的人生相似，但策略模式围绕可以互换的算法创建了极其成功的业务，而状态走的可能是更崇高的路，帮助对象通过改变它们的内部状态来控制其行为。不过你会发现，尽管路线不同，底层的设计是一样的。怎么回事？正如你将看到的，策略和状态意图差别很大。首先，我们来深入挖掘看看状态模式到底是什么，然后在本章末尾再回来探索它们的关系。

va Jaw 粉碎机

糖果机已经高科技化。没错，主流制造商发现，通过把CPU放进他们的糖果机，就可以提升销量，通过网络监测库存，并且更精准地测算客户的满意度。

但这些制造商只是糖果机的专家，不是软件开发人员，他们向你求助：

至少这是他们的说法，我们认为，他们只是厌烦了老旧的技术，需要想办法让他们的工作更刺激。



万能糖果公司

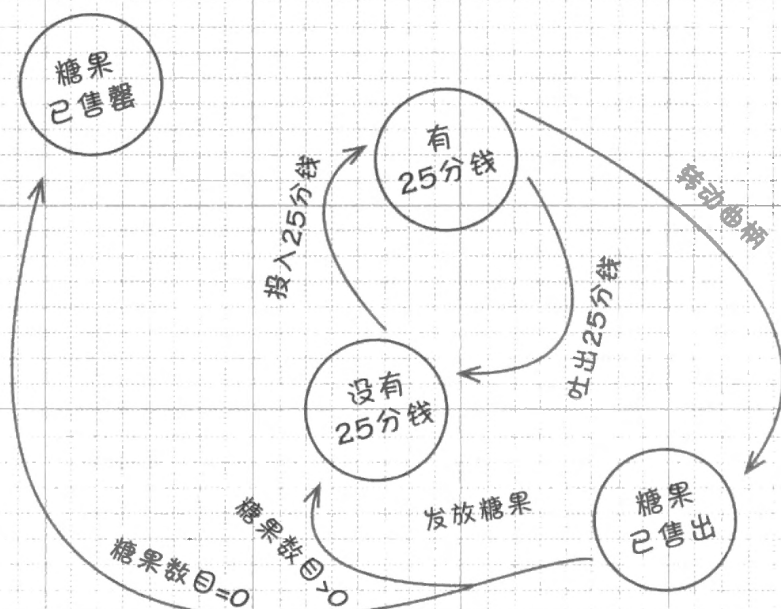
有糖果机的地方
永远充满活力

我们认为糖果机的控制器需要像下图一样工作。

我们希望你能用Java语言帮我们实现它！

将来我们可能会添加更多的行为，因此你需要让设计尽量保持弹性和可维护性！

——万能糖果工程师



办公室隔间对话



我们看看这张图，了解
万能糖果公司的这帮家伙
想要些什么……

Judy: 这张图看起来像一张状态图。

Joe: 没错，每个圆圈都是一个状态……

Judy: ……箭头是状态迁移。

Frank: 慢点，你们两个，我是很久以前学的状态图，快忘光了。你们能不能提醒我状态图是干什么的吗？

Judy: 当然可以，Frank。看这些圆圈，它们是状态。“没有25分钱”可能是糖果机的开始状态，因为它待在那里等着你投钱。所有状态都代表机器的不同配置和某种行动方式，需要某些动作把它们带到另一个状态。

Joe: 没错。看，要去往另一个状态，你需要做一些事情，像把25分硬币放入机器。看到从“没有25分钱”到“有25分钱”的箭头了吗？

Frank: 看到……

Joe: 这意味着如果糖果机在“没有25分钱”状态，你放入25分钱，它会改变到“有25分钱”状态。这就是状态迁移。

Frank: 啊，我懂了！如果我是在“有25分钱”状态，我可以转动曲柄改变到“糖果已售出”状态，或者退币转回到“没有25分钱”状态。

Judy: 答对！

Frank: 看来没那么糟。显然，我们有四个状态，我认为我们也有四个动作“投入25分钱”“吐出25分钱”“转动曲柄”和“发放”。不过……当我们发放时，我们在“糖果已售出”状态测试零或更多糖果，然后要么去往“糖果已售罄”状态，要么去往“没有25分钱”状态。因此，实际上我们有五个状态迁移。

Judy: 测试零或更多糖果，也意味着我们也要跟踪糖果的数目。任何时候机器给出一颗糖果，都有可能是最后一颗，如果是，我们需要迁移到“糖果已售罄”状态。

Joe: 还有，别忘了你可以做没有意义的事，像糖果机在“没有25分钱”状态下尝试吐出25分钱，或者同时投入两个25分钱。

Frank: 噢，我没想到这个，我们也要关心这些。

Joe: 对每个可能的动作，我们都要检查，看看我们所处的状态，以及动作是否合适。我们可以做到！让我们开始把状态图映射到代码……

状态机101

我们如何从状态图得到真正的代码？下面简单介绍如何实现状态机。

1 首先，收集状态。



2 接下来，创建一个实例变量来持有当前状态，并为每个状态定义值。

我们把“糖果已售罄”简称“已售罄”。

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;
```

每一个状态用一个不同的整数代表……

```
int state = SOLD_OUT;
```

……这是一个实例变量，持有当前状态。我们把它设置为“已售罄”，因为糖果机一开始拆箱并装机时，是没有装糖果的。

3 现在我们收集系统中可能发生的所有动作。

投入25分钱

转动曲柄

吐出25分钱

发放

这些动作是糖果机的界面——你能对糖果机做的事情。

看这个图，调用这些动作的任何一个都会导致状态迁移。

发放更多时候是糖果机自己调用自己的内部动作。

- 4 现在我们创建一个类，它扮演状态机的角色。我们为每个动作创建一个方法，这些方法使用条件语句来决定每个状态下合适的行为。例如，对于“投入25分钱”动作，我们可能编写一个这样的方法：

```
public void insertQuarter() {
```

```
    if (state == HAS_QUARTER) {
```

每个可能的状态用
条件语句检查……

```
        System.out.println("You can't insert another quarter");
```

```
    } else if (state == NO_QUARTER) {
```

……为每个可能的状态展现
适当的行为……

```
        state = HAS_QUARTER;
```

```
        System.out.println("You inserted a quarter");
```

```
    } else if (state == SOLD_OUT) {
```

……也可以迁移到其他状态，
就像图中所描绘的。

```
        System.out.println("You can't insert a quarter, the machine is sold out");
```

```
    } else if (state == SOLD) {
```

```
        System.out.println("Please wait, we're already giving you a gumball");
```

```
    }
```

```
}
```

我们这里谈论的是，建模对象内状态的常用技巧。通过创建一个实例变量来持有状态值并在方法内编写条件代码来处理各种状态。

〇〇



在这段快速回顾之后，我们来实现糖果机！

编写代码

是时候实现糖果机了。我们知道，我们打算让一个实例变量来持有当前状态。然后，我们需要处理所有动作、行为和可能发生的状态迁移。对于动作，我们需要实现投入25分钱、移除25分钱、转动曲柄和发放糖果。我们也要实现检查糖果机是否为空。

```
public class GumballMachine {
```

```
    final static int SOLD_OUT = 0;
    final static int NO_QUARTER = 1;
    final static int HAS_QUARTER = 2;
    final static int SOLD = 3;
```

```
    int state = SOLD_OUT;
    int count = 0;
```

```
    public GumballMachine(int count) {
        this.count = count;
        if (count > 0) {
            state = NO_QUARTER;
        }
    }
}
```

现在我们开始把这些动作
实现为方法……

```
    public void insertQuarter() {
        if (state == HAS_QUARTER) {
            System.out.println("You can't insert another quarter");
        } else if (state == NO_QUARTER) {
            state = HAS_QUARTER;
            System.out.println("You inserted a quarter");
        } else if (state == SOLD_OUT) {
            System.out.println("You can't insert a quarter, the machine is sold out");
        } else if (state == SOLD) {
            System.out.println("Please wait, we're already giving you a gumball");
        }
    }
}
```

如果顾客刚买了糖果，他需要
稍等一下，直到事务做完，才
能投另一个25分钱。

这里有四个状态，符合万能糖果公司
的状态图。

这个实例变量跟踪当前状态，
从已售罄状态开始。

我们有第二个实例变量，用来
跟踪机器内的糖果数目。

构造器需要初始糖果库存作为参数。
如果库存不为零，机器进入状态“没
有25分钱”，意思是，它正在等着别
人投入25分钱。否则，它停留在“已
售罄”状态。

当投入25分钱……

……如果已投入过25分
钱，我们告诉顾客……

……否则，我们接受25
分钱，迁移到“有25分
钱”状态。

如果糖果已售罄，
拒绝收钱。

```

public void ejectQuarter() {
    if (state == HAS_QUARTER) {
        System.out.println("Quarter returned");
        state = NO_QUARTER;
    } else if (state == NO_QUARTER) {
        System.out.println("You haven't inserted a quarter");
    } else if (state == SOLD) {
        System.out.println("Sorry, you already turned the crank");
    } else if (state == SOLD_OUT) {
        System.out.println("You can't eject, you haven't inserted a quarter yet");
    }
}

public void turnCrank() {
    if (state == SOLD) {
        System.out.println("Turning twice doesn't get you another gumball!");
    } else if (state == NO_QUARTER) {
        System.out.println("You turned but there's no quarter");
    } else if (state == SOLD_OUT) {
        System.out.println("You turned, but there are no gumballs");
    } else if (state == HAS_QUARTER) {
        System.out.println("You turned...");
        state = SOLD;
        dispense();
    }
}

public void dispense() {
    if (state == SOLD) {
        System.out.println("A gumball comes rolling out the slot");
        count = count - 1;
        if (count == 0) {
            System.out.println("Oops, out of gumballs!");
            state = SOLD_OUT;
        } else {
            state = NO_QUARTER;
        }
    } else if (state == NO_QUARTER) {
        System.out.println("You need to pay first");
    } else if (state == SOLD_OUT) {
        System.out.println("No gumball dispensed");
    } else if (state == HAS_QUARTER) {
        System.out.println("You need to turn the crank");
    }
}

// 其他方法放这里, 像toString()和refill()

```

现在, 如果顾客尝试移除25分钱……

……如果有25分钱, 就把钱退出来, 回到“没有25分钱”状态。

……否则, 没有25分钱, 我们无法退钱。

如果糖果售罄, 就不能接受25分钱, 当然也不能退钱!

如果顾客转动曲柄……

有人尝试欺骗机器。

我们首先需要 一个25分钱。

我们不能交付糖果, 因为没有。

成功! 拿到糖果了。改变状态到“已售出”, 并调用机器的dispense()方法。

调用此方法, 发放糖果。

我们在“已售出”状态, 给他们糖果!

我们在这里处理“糖果已售罄”的情况。如果这是最后一颗, 我们就把机器的状态设置为“已售罄”。否则, 就回到“没有25分钱”。

这些都不应该发生, 但如果顾客这么做了, 我们给出错误消息, 而不是糖果。

内部测试

感觉我们用考虑周密的方法学做出了坚实的设计，你不觉得吗？在我们把它交给万能糖果公司，安装到实际的糖果机之前，我们来做一个小小的内部测试。以下是测试程序：

```
public class GumballMachineTestDrive {

    public static void main(String[] args) {
        GumballMachine gumballMachine = new GumballMachine(5);

        System.out.println(gumballMachine);
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        System.out.println(gumballMachine);
        gumballMachine.insertQuarter();
        gumballMachine.ejectQuarter();
        gumballMachine.turnCrank();
        System.out.println(gumballMachine);
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.ejectQuarter();
        System.out.println(gumballMachine);
        gumballMachine.insertQuarter();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        System.out.println(gumballMachine);
    }
}
```

加载时放五颗糖果。

打印出机器的状态。

扔一个25分进去……

转动曲柄；我们应该得到糖果。

再次打印出机器的状态。

投入一个25分钱……

要求退回。

转动曲柄，我们不应该拿到糖果。

再次打印出机器的状态。

投入25分钱……

转动曲柄，我们应该拿到糖果。

投入25分钱……

转动曲柄；我们应该拿到糖果。

在没投钱时要求退钱。

再次打印出机器的状态。

投入两个25分钱……

转动曲柄；我们应该拿到糖果。

现在做压力测试…… 😊

再次打印机器的状态。

File Edit Window Help mightygumball.com

%java GumballMachineTestDrive

Mighty Gumball, Inc.

Java-enabled Standing Gumball Model #2004

Inventory: 5 gumballs

Machine is waiting for quarter

You inserted a quarter

You turned...

A gumball comes rolling out the slot

Mighty Gumball, Inc.

Java-enabled Standing Gumball Model #2004

Inventory: 4 gumballs

Machine is waiting for quarter

You inserted a quarter

Quarter returned

You turned but there's no quarter

Mighty Gumball, Inc.

Java-enabled Standing Gumball Model #2004

Inventory: 4 gumballs

Machine is waiting for quarter

You inserted a quarter

You turned...

A gumball comes rolling out the slot

You inserted a quarter

You turned...

A gumball comes rolling out the slot

You haven't inserted a quarter

Mighty Gumball, Inc.

Java-enabled Standing Gumball Model #2004

Inventory: 2 gumballs

Machine is waiting for quarter

You inserted a quarter

You can't insert another quarter

You turned...

A gumball comes rolling out the slot

You inserted a quarter

You turned...

A gumball comes rolling out the slot

Oops, out of gumballs!

You can't insert a quarter, the machine is sold out

You turned, but there are no gumballs

Mighty Gumball, Inc.

Java-enabled Standing Gumball Model #2004

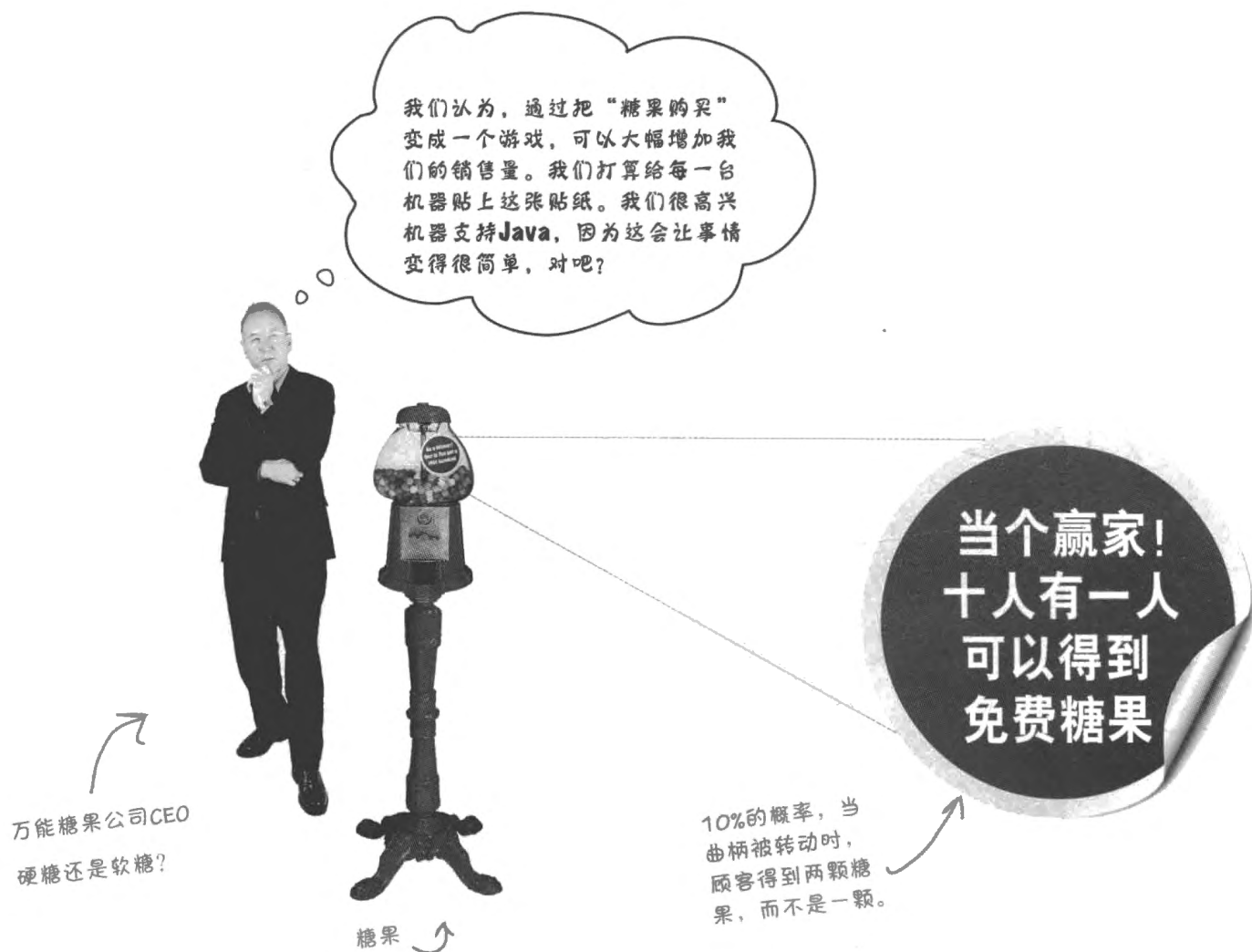
Inventory: 0 gumballs

Machine is sold out

你知道它会来的……变更请求！

万能糖果公司已经把你的代码加载进他们的新机器，他们的QA专家正在测试。目前为止，在他们看来一切顺利。

事实上，实在是太顺利了，因此他们想往前迈一步……

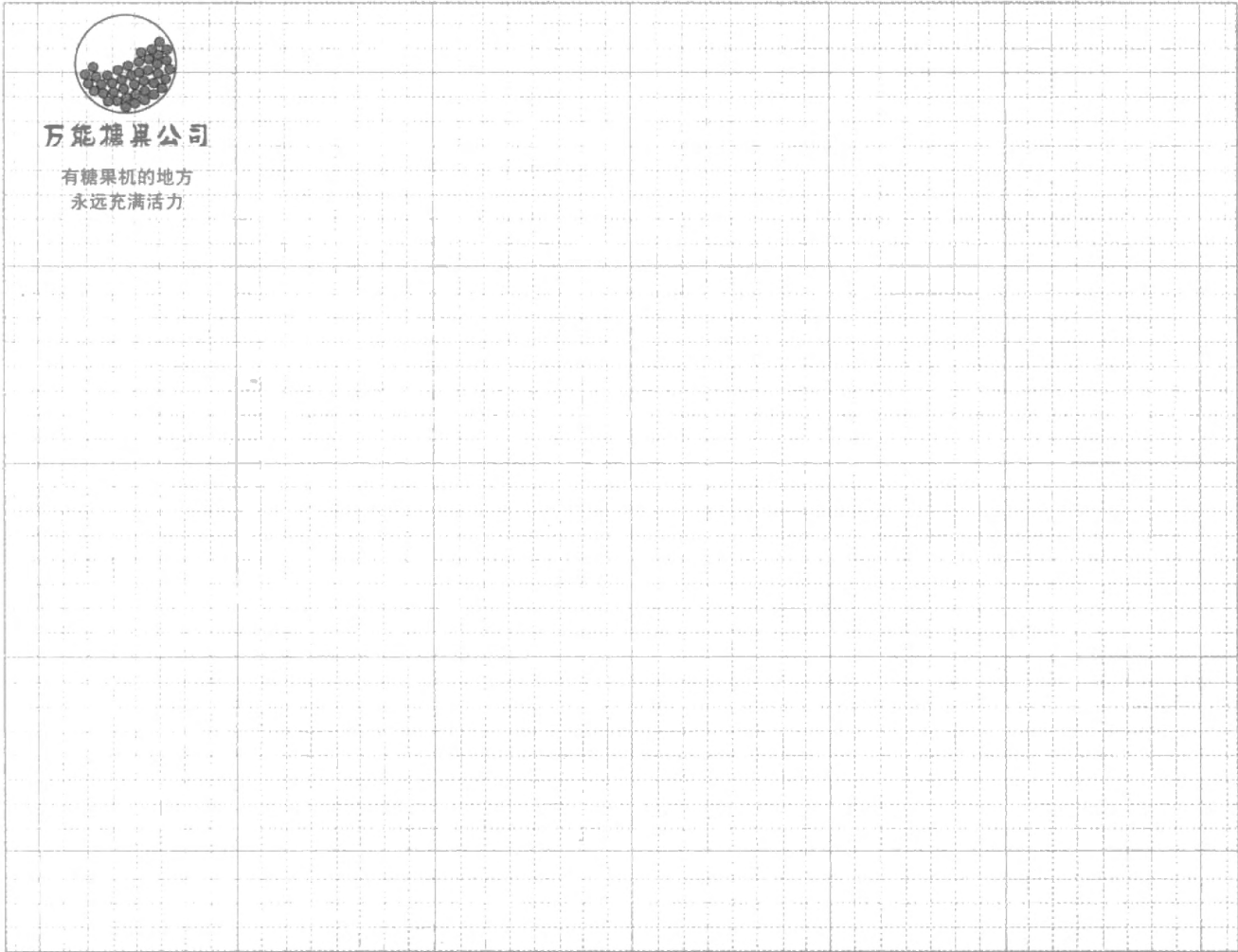




设计谜题

为糖果机画一张状态图，处理10中1的游戏。

在这个游戏中，有10%的概率“已售出”状态会导致掉下两颗糖果，而不是一颗。在你继续下一步之前，请把你的答案和我们的（在本章的最后）做比对，确定我们的看法一致……



用万能糖果公司的文具来画你的状态图。

乱糟糟的状态……

只是因为你用考虑周密的方法学编写糖果机，不意味着代码容易扩展。事实上，当你回顾你的代码并考虑必须修改的地方时……

```
final static int SOLD_OUT = 0;
final static int NO_QUARTER = 1;
final static int HAS_QUARTER = 2;
final static int SOLD = 3;
```

首先，你必须在这里添加一个新的“赢家”（WINNER）状态，这不算太糟……

```
public void insertQuarter() {
    // 投入25分钱的代码在这里
}
```

```
public void ejectQuarter() {
    // 吐出25分钱的代码在这里
}
```

……但是然后呢，你不得不在每个方法中添加一个新的条件判断，以处理“赢家”状态，有许多代码要改。

```
public void turnCrank() {
    // 转动曲柄的代码在这里
}
```

turnCrank()会变得特别凌乱，因为你不得不添加代码来检查目前的顾客是否赢家，然后切换到“赢家”状态或“已售出”状态。

```
public void dispense() {
    // 发放的代码在这里
}
```



削尖你的铅笔

以下哪些选项描述了我们实现的状态？（多选）

- ☐ A. 这份代码肯定没有遵守开放-关闭原则。
- ☐ B. 这份代码会让FORTRAN程序员感到骄傲。
- ☐ C. 这个设计不是很面向对象。
- ☐ D. 状态迁移不明显，被埋藏在一堆条件语句中。
- ☐ E. 我们没有封装变化的部分。
- ☐ F. 未来添加的代码有可能导致bug。



OK, 这可不好。

我认为我们的第一个版本很棒,但随着时间的推移,万能糖果公司不断要求新行为,这个版本快顶不住了。**bug**的概率大增会给我们带来麻烦,更不用说**CEO**会把我们逼疯了。

Frank: 你说得没错! 我们需要重构这份代码,好让它容易维护和修改。

Judy: 我们真应该尝试局部化每个状态的行为,这样,如果我们对某个状态做了改变,不会有把其他的代码搞乱的风险。

Frank: 没错,换句话说,遵循“封装变化”原则。

Judy: 正是如此。

Frank: 如果我们把每个状态的行为放进自己的类,那么每个状态只要实现它自己的动作就可以了。

Judy: 正确。也许糖果机只需要委托代表当前状态的状态对象。

Frank: 啊,你说得好,多用组合……原则的应用。

Judy: 帅。我没有100%确定怎么做,但我想我们在正确的方向上了。

Frank: 我不知道这是否可以使得添加新的状态更容易?

Judy: 我认为是这样……我们还是得改变代码,但是改变的范围会小得多,因为添加一个新的状态意味着我们不得不添加一个新的类,也许改变一些迁移。

Frank: 听起来很合我心意。我们来开始推敲这个新设计吧!

新的设计

看起来我们有了一个新的计划，不维护已有代码，我们打算重写，把状态对象封装进它们自己的类，然后在动作发生时委托给当前状态。

在这里，我们遵循我们的设计原则，因此最后应该得到一个更容易维护的设计。我们打算这样做：

- ❶ 首先，我们打算定义一个State接口，里面针对糖果机内的每个动作有一个方法。
- ❷ 然后我们打算为机器的每个状态实现一个State类。这些类将负责在相应状态下的机器行为。
- ❸ 最后，我们打算去除所有条件代码，把工作委托给State类。

我们不仅遵循设计原则，正如你将看到的，实际上我们实现了状态模式。不过，在重新编写我们的代码后，我们再学习官方的状态模式……

现在我们打算把一个状态的所有行为放进一个类。这样，我们就把行为局部化，使得改变和理解容易了许多。



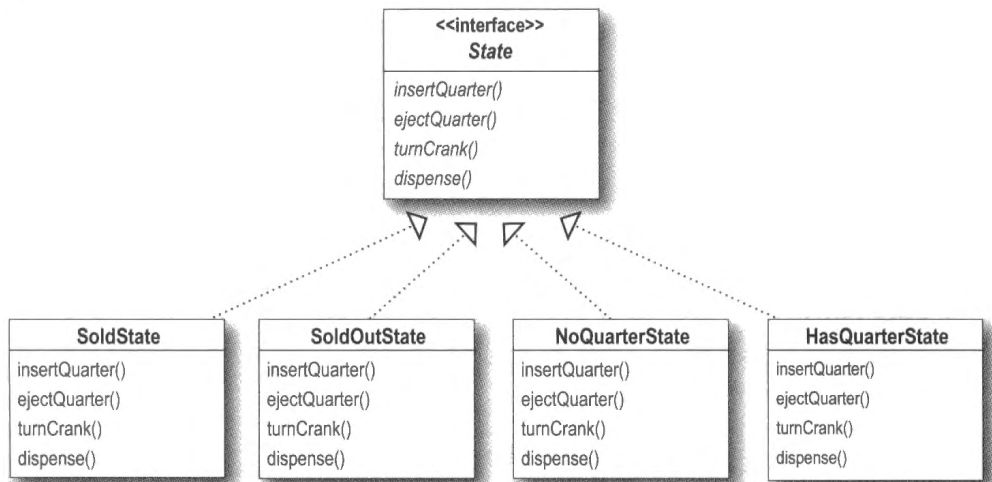
定义State接口和类

首先我们来为State创建一个接口，所有状态都要实现它：

这是所有状态的接口。这些方法（和之前代码一样的方法）都直接映射到糖果机上发生的动作。

然后，把设计中的每个状态封装进一个实现State接口的类。

我们看之前的代码，找出我们需要哪些状态……



……我们把每个状态直接映射到一个类。

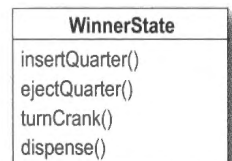
```

public class GumballMachine {

    final static int SOLD_OUT = 0;
    final static int NO_QUARTER = 1;
    final static int HAS_QUARTER = 2;
    final static int SOLD = 3;

    int state = SOLD_OUT;
    int count = 0;
}
  
```

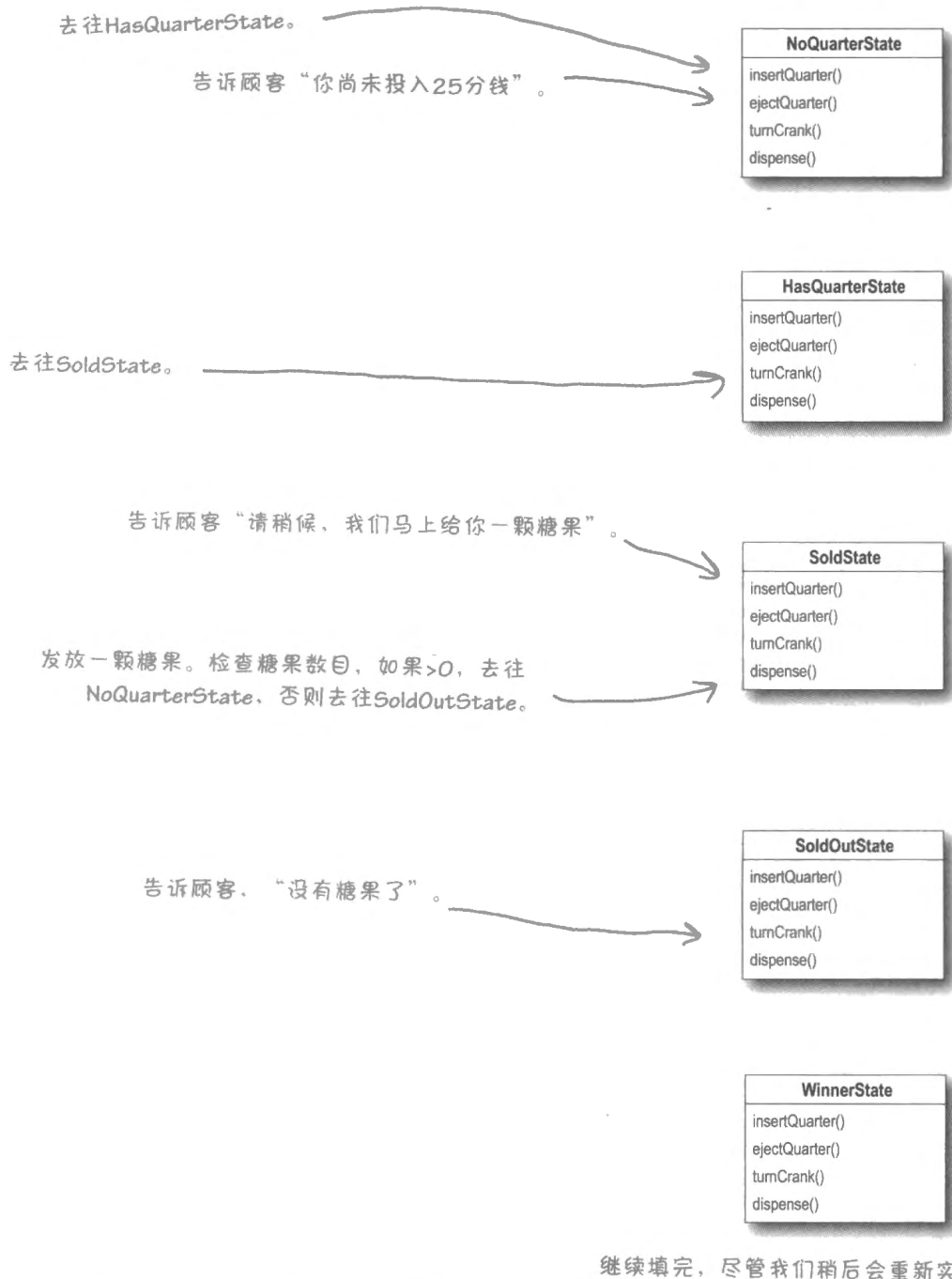
别忘了，我们需要一个新的“赢家”状态，它实现State接口。在我们重新实现第一个版本的糖果机之后，再回来处理这部分。





削尖你的铅笔

要实现我们的状态，我们首先需要指定当每个动作被调用时类的行为。请在下图为每个类的每个动作的行为加上注释，我们已经先帮你填写了几个。



实现State类

是时候实现状态了，我们知道了我们要什么行为，只需要把它变成代码。我们打算严格遵从所写的状态机代码，但这一次各种东西被打散进不同的类。

我们从NoQuarterState开始：

首先我们需要实现State接口。

我们通过构造器得到糖果机的引用，把它存放在实例变量中。

```
public class NoQuarterState implements State {
    GumballMachine gumballMachine;
```

```
    public NoQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }
```

如果有人投入一个25分钱，我们打印出一条消息，说25分钱已接受，然后改变机器的状态到HasQuarterState。

```
    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        gumballMachine.setState(gumballMachine.getHasQuarterState());
    }
```

一会你会看到这些如何工作……

```
    public void ejectQuarter() {
        System.out.println("You haven't inserted a quarter");
```

你不能把钱退回来，因为你没给钱！

```
    public void turnCrank() {
        System.out.println("You turned, but there's no quarter");
    }
```

```
    public void dispense() {
        System.out.println("You need to pay first");
    }
```

如果你不付钱，就不能得到糖果。

没有付钱，我们不能发放糖果。



重做糖果机

在我们完成State类之前，我们打算重做糖果机。这样，你可以看到一切是怎样配合运转的。我们从状态相关的实例变量开始，并把代码从使用整数改为使用状态对象：

```
public class GumballMachine {
```

```
    final static int SOLD_OUT = 0;  
    final static int NO_QUARTER = 1;  
    final static int HAS_QUARTER = 2;  
    final static int SOLD = 3;
```

```
    int state = SOLD_OUT;  
    int count = 0;
```

旧代码

在GumballMachine中，我们更新代码为使用新的类，而不再使用静态整数。两者的代码很相似，除了在一个类中是整数，而在另一个类中是其他对象……

```
public class GumballMachine {
```

```
    State soldOutState;  
    State noQuarterState;  
    State hasQuarterState;  
    State soldState;
```

```
    State state = soldOutState;  
    int count = 0;
```

新代码

所有State对象都是在构造器中创建并赋值。

这个现在持有一个State对象，而不是整数。

现在，我们来看看完整的GumballMachine类……

```

public class GumballMachine {

    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;

    State state;
    int count = 0;

    public GumballMachine(int numberGumballs) {
        soldOutState = new SoldOutState(this);
        noQuarterState = new NoQuarterState(this);
        hasQuarterState = new HasQuarterState(this);
        soldState = new SoldState(this);

        this.count = numberGumballs;
        if (numberGumballs > 0) {
            state = noQuarterState;
        } else {
            state = soldOutState;
        }
    }

    public void insertQuarter() {
        state.insertQuarter();
    }
    public void ejectQuarter() {
        state.ejectQuarter();
    }
    public void turnCrank() {
        state.turnCrank();
        state.dispense();
    }

    void setState(State state) {
        this.state = state;
    }

    void releaseBall() {
        System.out.println("A gumball comes rolling out the slot...");
        if (count > 0) {
            count = count - 1;
        }
    }
    // 更多方法在这里，包括每个状态的getter……
}

```

这里又是所有的State……

……和State实例变量。

count实例变量持有糖果数量，一开始机器是空的。

构造器取得糖果的初始数目，存储在一个实例变量中。

它也创建State实例。每个状态一个。

如果有超过0颗糖果，我们设置状态为NoQuarterState，否则，从SoldOutState开始。

现在动作很容易实现。我们只要委托到当前状态。

注意，在GumballMachine中，我们不需要为dispense()提供一个动作方法，因为它只是一个内部动作；用户不能直接要求机器发放糖果。不过，我们确实在State对象的turnCrank()方法中调用dispense()。

这个方法允许其他对象（像我们的State对象）把机器的状态迁移到各个状态。

这个机器支持一个releaseBall()的helper方法，该方法放出糖果，并将count实例变量的值减1。

这包括了以下方法：getNoQuarterState()，取得每个状态对象，getCount()，取得糖果数量。

实现更多状态

现在你开始感觉出来了，糖果机和状态是怎样配合的，我们来实现HasQuarterState和SoldState类……

当状态被实例化时，我们给它传入一个GumballMachine的引用，用来迁移机器到各个状态。

```
public class HasQuarterState implements State {
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        gumballMachine.setState(gumballMachine.getSoldState());
    }

    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```

这是状态的一个不恰当动作。

退回顾客的25分钱，迁移到NoQuarterState。

当曲柄被转动，我们通过调用其setState()方法，并传给它SoldState对象，把机器迁移到SoldState状态。SoldState对象通过getSoldState()的getter方法（每个状态都有一个getter方法）取得。

这个状态的另一
个不恰当动作。

现在，我们来看看SoldState类……

```
public class SoldState implements State {
    //构造器和实例变量在这里

    public void insertQuarter() {
        System.out.println("Please wait, we're already giving you a gumball");
    }

    public void ejectQuarter() {
        System.out.println("Sorry, you already turned the crank");
    }

    public void turnCrank() {
        System.out.println("Turning twice doesn't get you another gumball!");
    }
}
```

这是这个状态的所有不恰当的动作。

真正的工作在这里开始……

```
public void dispense() {
    gumballMachine.releaseBall();
    if (gumballMachine.getCount() > 0) {
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    } else {
        System.out.println("Oops, out of gumballs!");
        gumballMachine.setState(gumballMachine.getSoldOutState());
    }
}
}
```

我们在SoldState，意味着顾客已经付钱了，因此，我们首先需要请求机器放出糖果。

然后我们问机器糖果数目是多少，并迁移到NoQuarterState或SoldOutState。



回顾GumballMachine的实现。如果曲柄被转动，但是没有成功（例如顾客没有先投入25分钱），我们还是会调用dispense()，尽管没有必要。这怎么修正？



削尖你的铅笔

还剩下一个类没有实现：SoldOutState。为什么你不实现它呢？小心而全面地思考，在每种情况下，糖果机应该有什么样的行为。在继续下一页之前，先对一下你的答案……

```
public class SoldOutState implements _____ {
    GumballMachine gumballMachine;

    public SoldOutState(GumballMachine gumballMachine) {

    }

    public void insertQuarter() {

    }

    public void ejectQuarter() {

    }

    public void turnCrank() {

    }

    public void dispense() {

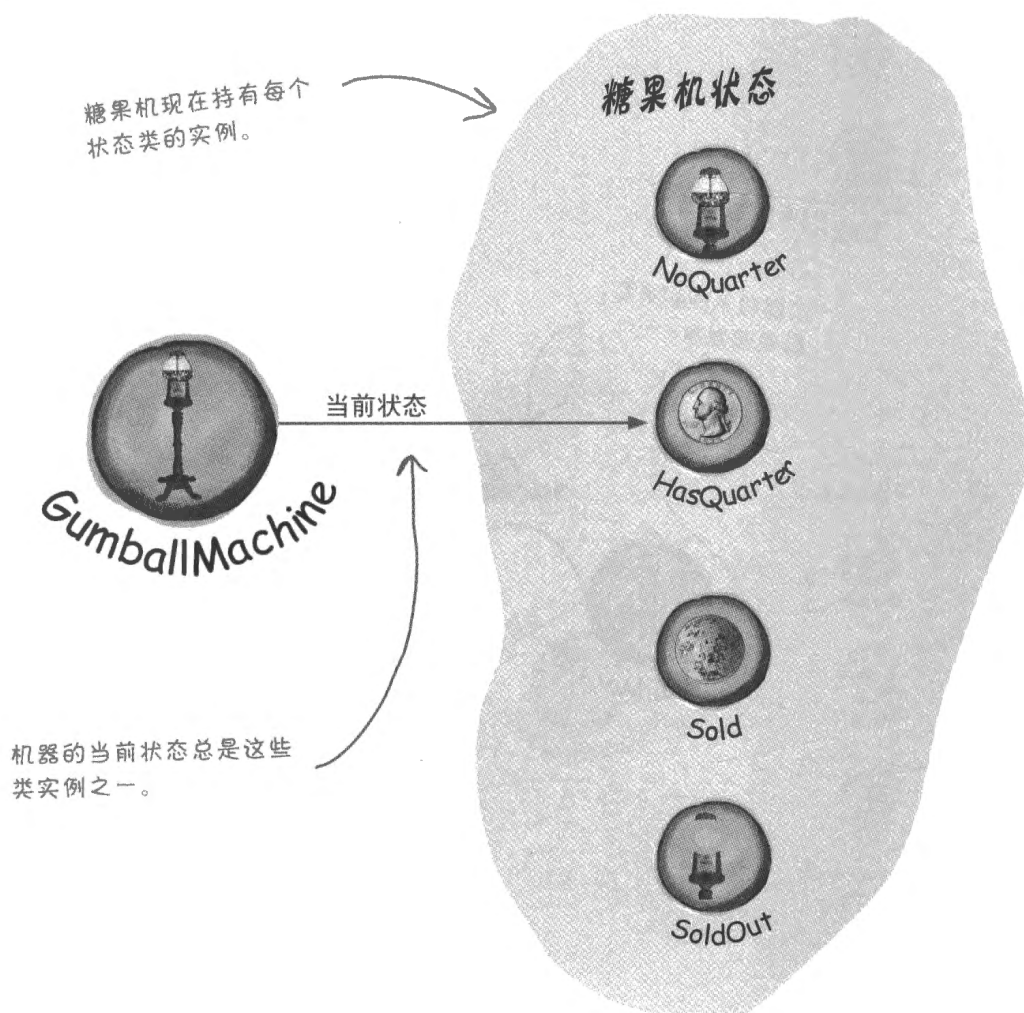
    }
}
```

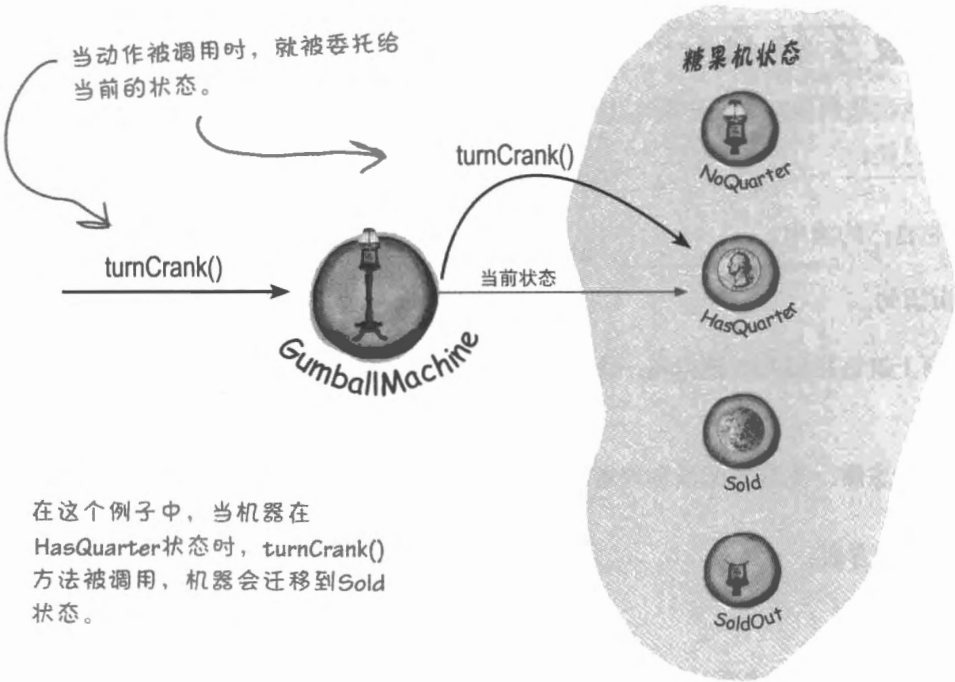
我们看看目前为止做了什么……

首先，你现在有一个糖果机实现。它在结构上和第一个版本差别很大，但功能上还是一样的。通过从结构上改变实现，你已经：

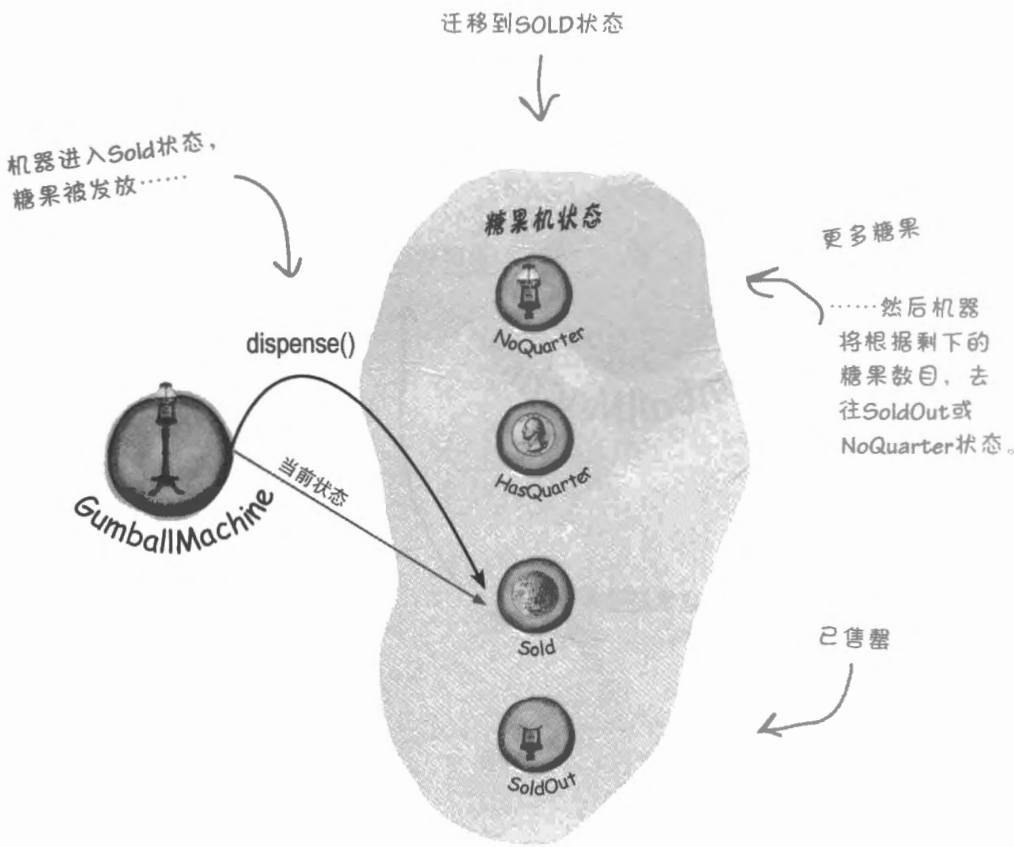
- 把每个状态的行为局部化到它自己的类中。
- 移除所有烦人、难以维护的if语句。
- 通过添加新的状态类（我们马上就这样做），使得每一个状态对修改关闭，糖果机对扩展开放。
- 创建更紧密映射万能糖果公司状态图、更容易阅读和理解的代码基以及类结构

现在我们来继续看我们已经做了的功能方面：





在这个例子中，当机器在HasQuarter状态时，turnCrank()方法被调用，机器会迁移到Sold状态。



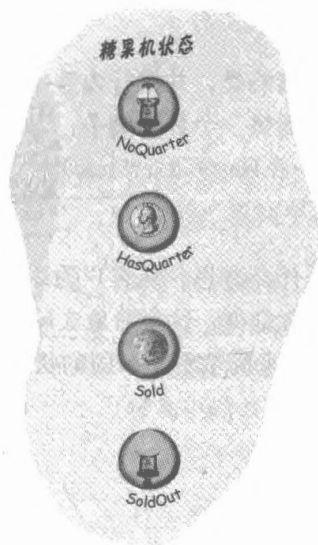
削尖你的铅笔

幕后花絮： 自助餐



跟踪糖果机的步骤，从NoQuarter状态开始。也请在图上标注机器的动作和输出。在这个练习中，你可以假设机器中有大量糖果。

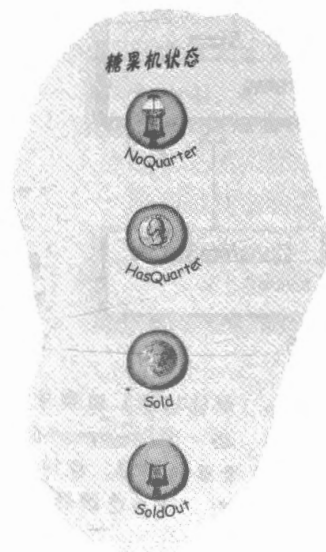
①



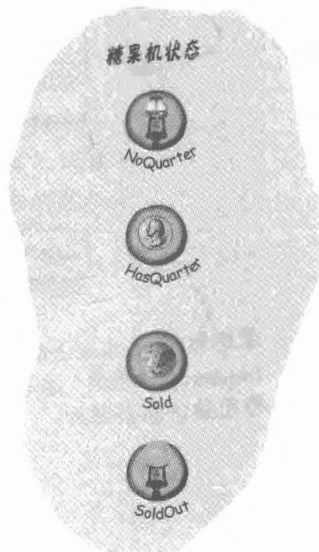
②



③



④



状态模式定义

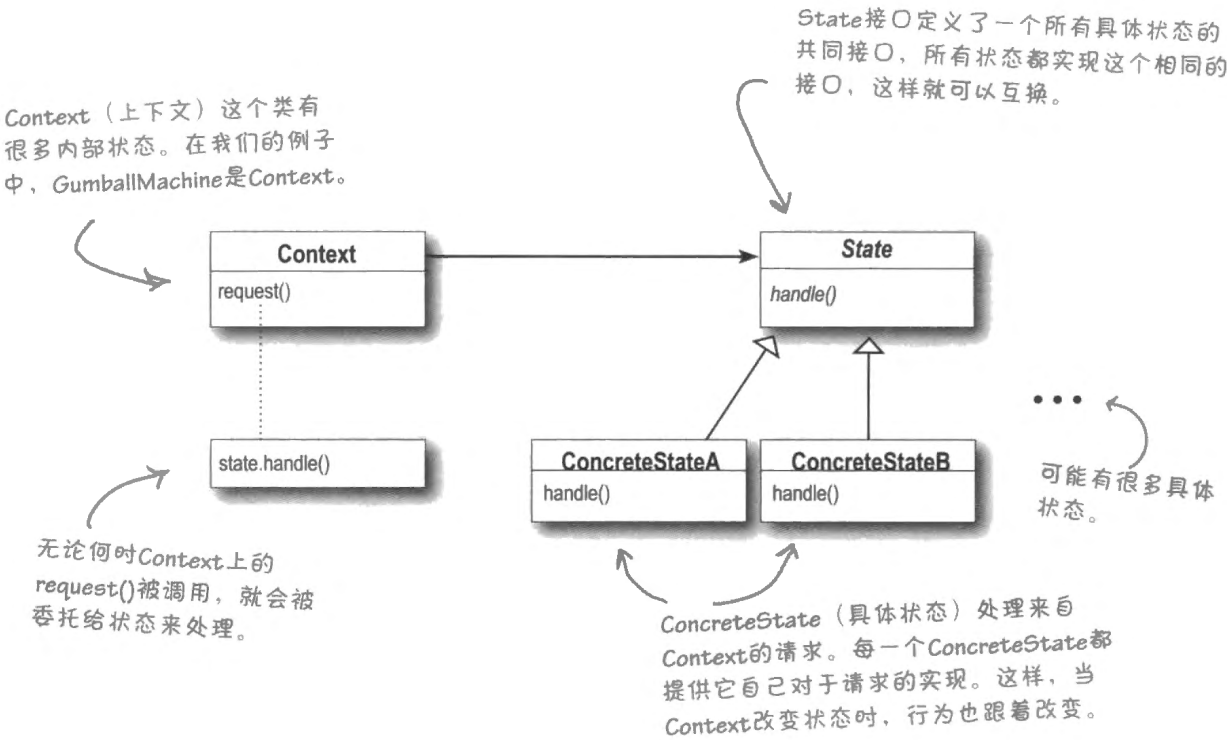
是的，这是真的，我们刚实现了状态模式！因此现在我们来看看什么是状态模式：

状态模式允许对象在内部状态改变时改变其行为。
对象看起来好像改变了它的类。

这个描述的第一部分含义丰富，是吧？因为这个模式把状态封装进分离的类，并把行为委托到代表当前状态的对象，我们知道行为随着内部状态而改变。糖果机提供一个好的例子：当糖果机在NoQuarterState时，你投入一个25分钱，机器会接受25分钱；在HasQuarterState时，你投入一个25分钱，行为就不同了，机器拒绝25分钱。

这个定义中的第二部分呢？一个对象“看起来好像改变了它的类”是什么意思？从客户的视角考虑：如果一个你正在使用的对象能够完全改变它的行为，那么你会觉得，这个对象实际上是从另一个类实例化而来的。但是，实际上你知道我们是在使用组合来简单引用不同的状态对象，造成类改变的假象。

OK，现在是时候来看看状态模式的类图了：





等一下，我记得策略模式，它的类图和这张类图一模一样。

你眼力真好（或者你看了该章开头的内容）！是的，类图本质上是一样的，但两个模式的区别在于意图。

对于状态模式，我们把一组行为封装进状态对象，context随时（把责任）委托给这些状态之一。随着时间的推移，当前状态不断在各个状态对象之间改变，以反映context的内部状态，因此，context的行为也随着时间的推移而改变。客户通常对状态对象知道得很少，甚至不知道。

对于策略，客户通常指定context要组合的策略对象。现在，虽然模式提供了弹性，可以在运行时改变策略对象，但对于一个context对象来说，经常有一个最适合的策略对象。例如，在第1章中，有些鸭子（例如绿头鸭）被配置成用典型的飞行行为来飞行，而其他鸭子（例如橡皮鸭和诱饵鸭）被配置成紧贴地面的飞行行为。

一般来说，把策略模式想成子类化的一个弹性替代方案；如果你使用继承定义了一个类的行为，那么你就和这个行为粘在一起了，很难改变。有了策略，你可以通过组合不同的对象改变行为。

把状态模式想成在context中放进许多条件的一个替代方案，通过把行为封装进状态对象，你可以简单地在context中改变状态对象来改变其行为。

没有蠢问题

问：在GumballMachine中，状态决定下一个状态应该是什么。是否ConcreteState总是决定接下来去往哪个状态？

答：不，不总是这样。也可以让Context决定状态迁移的流动。

一般的指南是，当状态迁移固定时，就适合放进Context中；但是，当迁移更加动态时，通常就会放进状态类自身（例如，在GumballMachine中，迁移到NoQuarter还是SoldOut的选择，依赖于运行时糖果的数量）。

把状态迁移放进状态类中的缺点是：造成状态类之间的依赖。在我们的GumballMachine实现中，我们尝试通过使用Context上的getter方法把依赖最小化，而不是显式硬编码具体状态类。

注意，通过做这个决策，你也相当于在决策：当系统进化时，哪个类对修改关闭（Context还是状态类）。

问：客户会直接和状态交互吗？

答：不会。状态是由Context使用，代表其内部状态和行为，因此对状态的所有请求都来自Context。客户不直接改变Context的状态。监督状态是Context的工作，通常你不会让客户在没有Context的知识的情况下，改变Context的状态。

问：如果在我的应用中，Context有许多实例，这些实例之间有可能共享状态对象吗？

答：是的，绝对可以，事实上这是很常见的场景。唯一的要求是你的状态对象不维护它们自己内部的context，否则，对于每个context，你需要独一无二的实例。

要共享状态，通常把每个状态分配到一个静态实例变量。如果你的状态需要使用Context中的方法或实例变量，每个handler()方法中，你也不得不给它一个到Context的引用。

问：看起来，使用状态模式总是增加设计中类的数目。看，我们的GumballMachine比原始设计多了多少类！

答：你说得对，通过把状态行为封装进分离的状态类，在设计中总得到更多的类。这通常是为了弹性所付出的代价。除非你的代码是“一锤子”的实现，用完就丢，否则往后你可能就会感谢这些额外添加的类。注意，重要的通常是暴露给客户的类数目，有一些办法可以把这些额外的类对客户隐藏（例如，声明它们为package private）。

还有，考虑另一种做法：如果你有一个应用，它有很多状态，你决定不使用分离的对象，而是用巨大的、整块的条件语句。这会使得你的代码难以维护和理解。通过使用对象，让状态变得显式，减少了理解和维护代码的工作量。

问：状态模式类图展示，State是一个抽象类。为什么你不在糖果机的状态实现中使用接口？

答：是的。假设我们没有共同的功能可以放进抽象类中，就会使用接口。在你自己的实现中，你可能要考虑用抽象类，好处是允许你在后面添加方法，而不会打破具体状态实现。

我们还是需要完成糖果机10中1游戏

记住，我们还没搞定。我们还有一个游戏要实现，但既然我们已经实现了状态模式，这应该很容易。

首先，我们需要给GumballMachine类添加一个状态：

```
public class GumballMachine {
```

```
    State soldOutState;
    State noQuarterState;
    State hasQuarterState;
    State soldState;
    State winnerState;
```

你只需要在这里添加一个新的 WinnerState，并在构造器中初始化它。

```
    State state = soldOutState;
    int count = 0;
    // 方法在这里
```

别忘了，你也得为 WinnerState添加一个getter方法。

```
}
```

现在我们来实现WinnerState类，和SoldState类很像：

```
public class WinnerState implements State {
```

```
    // 实例变量和构造器
    // insertQuarter错误信息
    // ejectQuarter错误信息
    // turnCrank错误信息
```

就像SoldState。

```
    public void dispense() {
```

```
        gumballMachine.releaseBall();
```

```
        if (gumballMachine.getCount() == 0) {
```

```
            gumballMachine.setState(gumballMachine.getSoldOutState());
```

```
        } else {
```

```
            gumballMachine.releaseBall();
```

如果还有第二颗糖果，就放出。

```
            System.out.println("YOU'RE A WINNER! You got two gumballs for your quarter");
```

```
            if (gumballMachine.getCount() > 0) {
```

```
                gumballMachine.setState(gumballMachine.getNoQuarterState());
```

```
            } else {
```

```
                System.out.println("Oops, out of gumballs!");
```

```
                gumballMachine.setState(gumballMachine.getSoldOutState());
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

在这里，我们放出两颗糖果，然后去往 NoQuarterState或SoldOutState。

如果我们能够放出两颗糖果，就让用户知道他是赢家。

完成游戏

我们只要再改一点：我们需要实现随机的机会，并添加一个到WinnerState的迁移。

我们打算把这两者都添加到HasQuarterState，因为顾客在这个状态转动曲柄：

```
public class HasQuarterState implements State {
    Random randomWinner = new Random(System.currentTimeMillis());
    GumballMachine gumballMachine;

    public HasQuarterState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert another quarter");
    }

    public void ejectQuarter() {
        System.out.println("Quarter returned");
        gumballMachine.setState(gumballMachine.getNoQuarterState());
    }

    public void turnCrank() {
        System.out.println("You turned...");
        int winner = randomWinner.nextInt(10);
        if ((winner == 0) && (gumballMachine.getCount() > 1)) {
            gumballMachine.setState(gumballMachine.getWinnerState());
        } else {
            gumballMachine.setState(gumballMachine.getSoldState());
        }
    }

    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```

首先我们添加一个随机数生成器，生成10%赢的机会……

……然后我们决定这个顾客是否赢了。

如果赢了，而且有足够的剩余糖果可以让他一次取得两颗，去往WinnerState；否则，去往SoldState（就和平常一样）。

哇哦，实现起来相当简单！我们就是给GumballMachine添加了一个新的状态然后实现它。

接下来我们要做的，只是实现我们的机会游戏，并迁移到正确的状态。看起来我们的新代码策略奏效了……

为万能糖果公司的CEO演示

万能糖果公司的CEO来访，来看我们的新糖果机游戏代码的演示。希望这些状态都没问题！我们要让这个演示简短而甜蜜(CEO们的注意力之短众所周知)，但希望时间足够长到至少让我们赢一次。

这个代码其实根本没有改；我们只是把它缩短了一点。

```
public class GumballMachineTestDrive {
```

再一次，从糖果机装5颗糖果开始。

```
    public static void main(String[] args) {
```

```
        GumballMachine gumballMachine = new GumballMachine(5);
```

```
        System.out.println(gumballMachine);
```

```
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
```

我们希望能赢，因此一直投币并转动曲柄。我们时不时打印出糖果机的状态……

```
        System.out.println(gumballMachine);
```

```
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
        gumballMachine.insertQuarter();
        gumballMachine.turnCrank();
```

```
        System.out.println(gumballMachine);
```

```
    }
```

```
}
```

整个工程团队都在会议室外等待，想看看基于状态模式的设计是否能赢得青睐！！





```
File Edit Window Help Whenisagumballajawbreaker?
%java GumballMachineTestDrive
Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 5 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
A gumball comes rolling out the slot...
A gumball comes rolling out the slot...
YOU'RE A WINNER! You got two gumballs for your quarter

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 3 gumballs
Machine is waiting for quarter

You inserted a quarter
You turned...
A gumball comes rolling out the slot...
You inserted a quarter
You turned...
A gumball comes rolling out the slot...
A gumball comes rolling out the slot...
YOU'RE A WINNER! You got two gumballs for your quarter
Oops, out of gumballs!

Mighty Gumball, Inc.
Java-enabled Standing Gumball Model #2004
Inventory: 0 gumballs
Machine is sold out
%
```

没有蠢问题

问: 为什么我们需要WinnerState? 我们不能就让SoldState发放两颗糖果?

答: 很棒的问题。SoldState和WinnerState几乎一样, 除了WinnerState发放两颗糖果而不是一颗。你当然可以把发放两颗糖果的代码放进SoldState。当然, 这样做的缺点是, 一个状态类代表两个状态: 赢家的状态和不是赢家的状态。因此, 为了减少代码重复, 你牺牲了状态类中的清晰性。另一件要考虑的事情是, 你在前面章节中学到的原则: 单一责任原则。把WinnerState的责任放进SoldState, 你就给了SoldState两个责任。促销结束后会怎么样? 或者游戏的赌注变了呢? 因此, 必须要折中以得到设计决策。

好啊！大家做了很棒的工作。有了这个新游戏，我们的销量飞涨。你知道，我们也造汽水机，我想我们可以给这些自动贩卖机也装上这套游戏。既然机龄4年的糖果机可以，其他应该也可以吧？



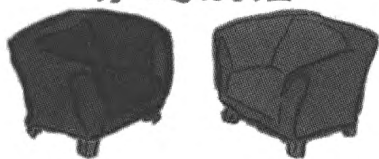
精神检查……

是的，万能糖果公司的CEO可能需要去做精神检查，但这不是我们在这里所要说的。在我们推出黄金版之前，我们来全面考虑一下，GumballMachine还有哪些方面要改进：

- 我们在Sold和Winning状态可能有许多重复的代码要清理。怎么做呢？我们把State做成一个抽象类，为方法内置一些缺省的行为；毕竟，像“你已经投入25分钱”这样的错误消息不会被顾客看见。因此，所有“错误响应”行为可以是通用的，继承自抽象的State类。
- 即使在没有25分钱时曲柄被转动，dispense()方法也会被调用，而机器正确地运作，不发放糖果，除非在正确的状态。通过让turnCrank()返回一个布尔值或者引入异常，我们可以轻易修正这一点。你认为哪个方案更好？
- 状态迁移的所有智能在State态类中，这可能导致什么问题？我们要将逻辑移进GumballMachine吗？有什么优缺点？
- 你会实例化许多GumballMachine对象吗？如果会，你可能想要把状态实例移到静态实例变量中并共享。这需要对GumballMachine和State做什么改变？

该死的Jim，我是糖果机，不是计算机！

炉边谈话



今夜话题：策略和状态模式重聚。

策略：

嘿，兄弟。你听说过我在第1章就出现了吧？

我刚去帮了模板方法那帮家伙一个忙，它们要我帮它们结束它们的章节。那么，我高贵的兄弟找我干什么？

我不知道，你听起来像总是在复制我所做的事情，只是用了不同的词来描述。你想想：我允许对象通过组合和委托整合不同的行为或算法。你只是在抄袭我。

是吗？怎么说？我不理解。

是的，那是一些精细的工作……我肯定你能明白这比继承行为更有威力的道理，对吧？

抱歉，你得解释一下这一点。

状态：

是的，肯定传得沸沸扬扬了。

一如既往，帮类在不同状态中展现不同的行为。

我承认我们所做的事情肯定有关系，但我的意图和你完全不同。我教客户使用组合和委托的方式是完全不一样的。

嗯，你如果多花一点时间考虑一下别人，你可能就理解了。无论如何，想想你是怎么工作的：你有一个正在实例化的类，通常你给它一个实现某些行为的策略对象，像第1章你处理嘎嘎叫行为的那个，对吧？真正的鸭子得到真正的嘎嘎叫；橡皮鸭得到实际是吱吱叫的嘎嘎叫。

是的，当然。现在，考虑我的工作机制，它是完全不同的。

策略：

嘿，得了吧，我也可以在运行时改变行为，这正是组合的目的！

好吧，我承认，我没有激励我的对象拥有一组定义良好的状态迁移。事实上，我通常喜欢控制我的对象使用什么策略。

是，是，继续做你的白日梦吧，兄弟。你表现得好像和我一样是大模式，但要搞清楚，我第1章出场，他们把你卡到第10章才出来。我的意思是，有多少人真的把这本书看到第10章？

我的兄弟，你总是在做梦。

状态：

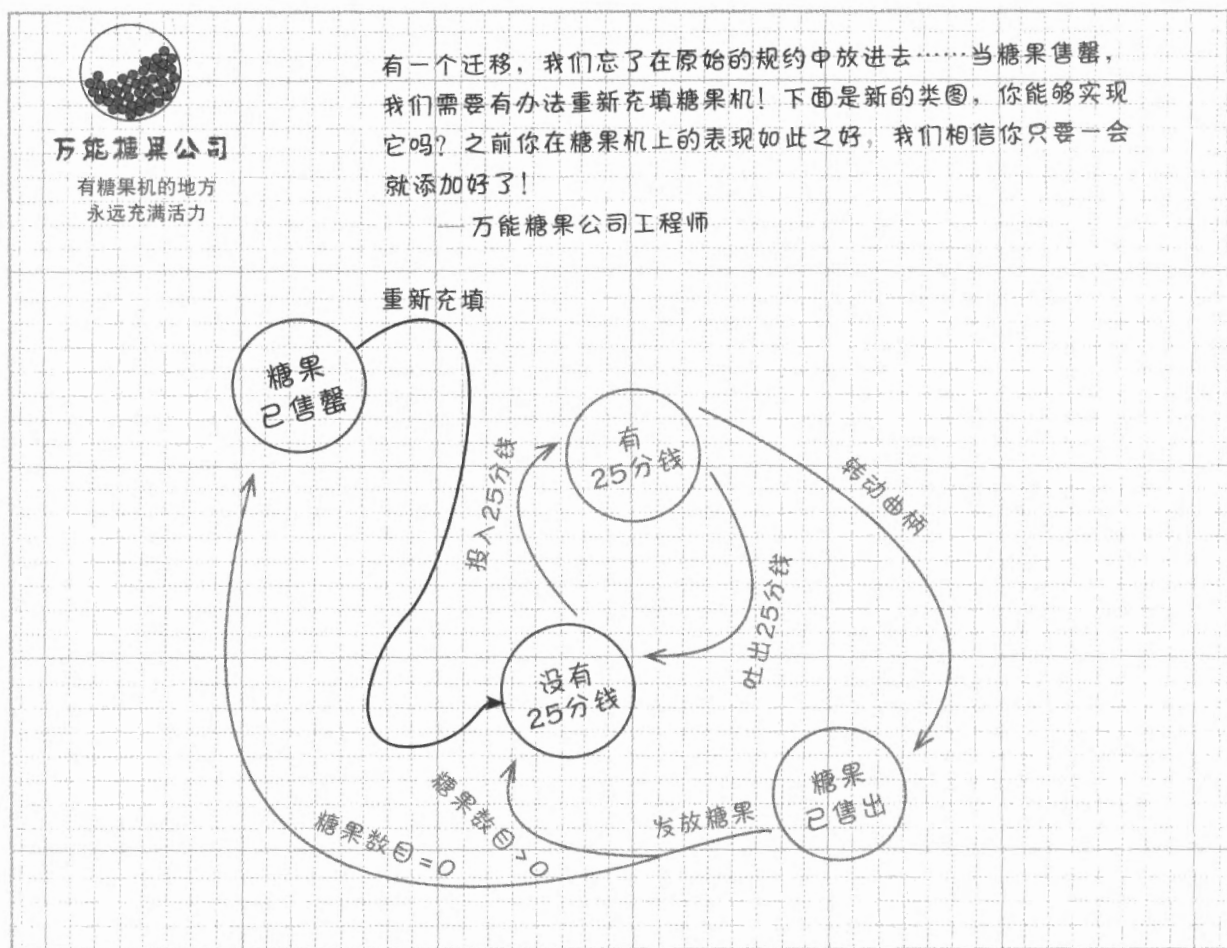
OK，当我的Context对象被创建之后，我可以告诉它们从什么状态开始，但随着时间的推移它们会改变自己的状态。

你当然可以，但是我的方式是围绕分离的状态建造；随着时间的推移，我的Context对象根据定义良好的状态迁移改变状态。换句话说，改变行为内置在我的方案中，这是我的工作机制！

看，我已经说过我们结构相似嘛，不过，我们做事物的意图是十分不同的。面对现实吧，这个世界同时需要用到我们两个。

开什么玩笑？这是Head First书籍，Head First的读者玉树临风，当然会读到第10章！

我们差点忘了！





削尖你的铅笔

我们需要你为糖果机编写`refill()`方法。此方法有一个参量（要添加到机器中的糖果数目），此方法应该更新糖果机内的糖果数目，并重置机器的状态。

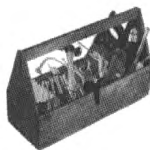
你的工作令人赞叹！我还有一些想法可以改变糖果工业，我需要你来实现它们。嘘！下一章我会告诉你这些想法。



WHO DOES WHAT?

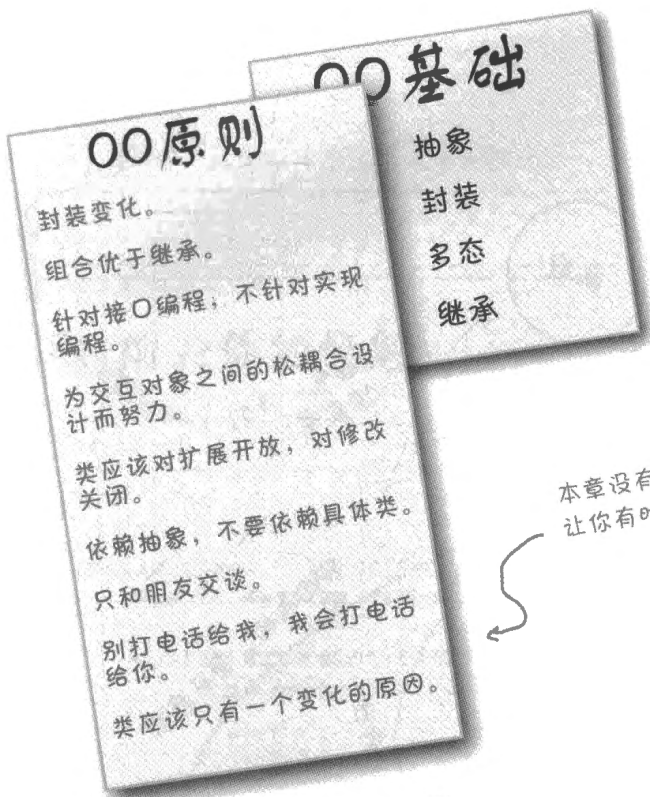
匹配每个模式及其描述：

模式	描述
状态	封装可以互换的行为，并使用委托来决定使用哪个行为。
策略	子类决定怎样实现算法的步骤。
模板方法	封装基于状态的行为并把行为委托给当前状态。



你的设计工具箱中的工具

又到了另一章的末尾，你已经有了足够的模式，可以轻松应对任何工作面试！



本章没有新的原则，让你有时间休息一下。

这是我们的新模式，如果你正在管理一个类中的状态，状态模式提供了封装状态的技巧。

状态，允许对象在内部状态改变时改变它的行为。对象看起来好像改变了它的类。

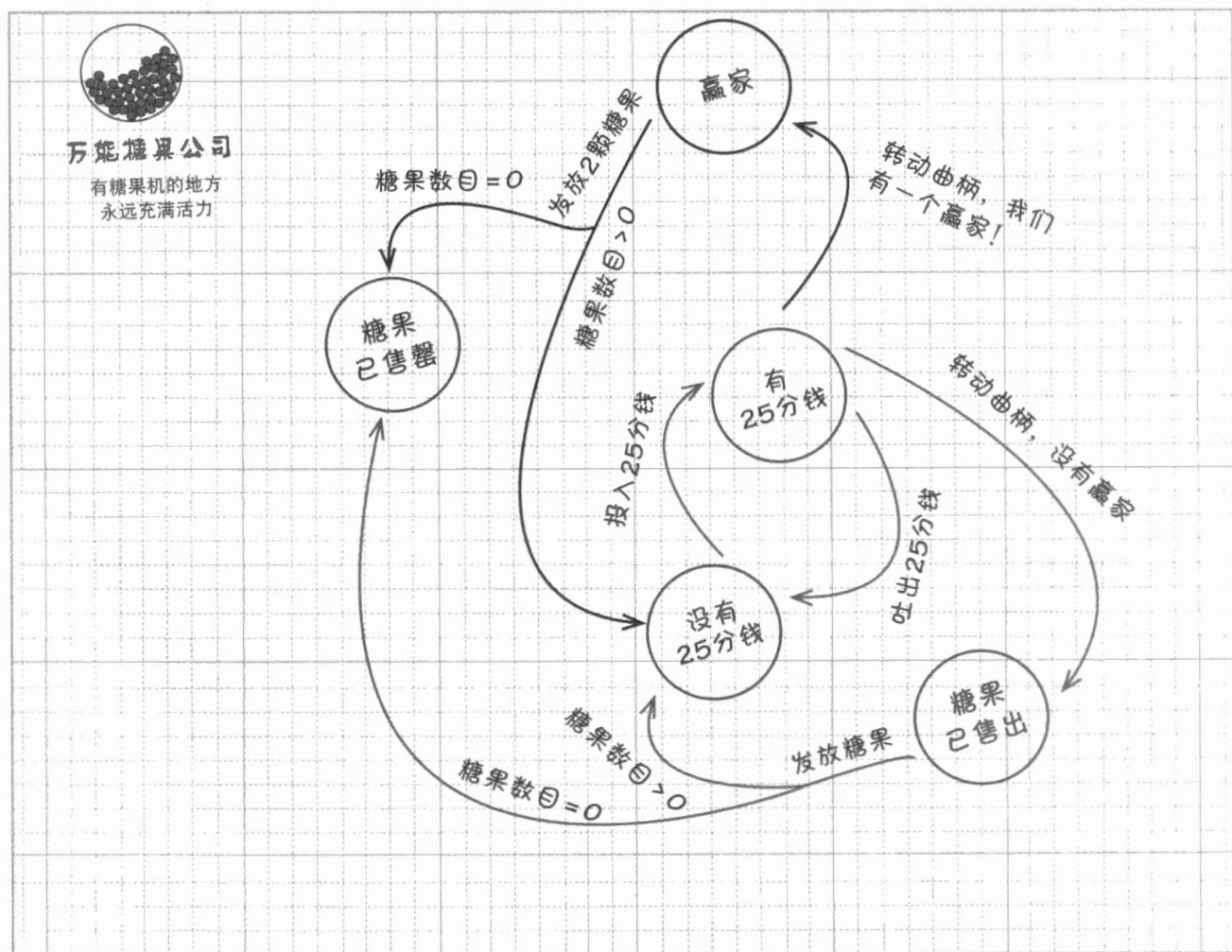
要点

- 状态模式允许一个对象基于内部状态拥有许多不同的行为。
- 和过程式状态机不同，状态模式用真正的类代表每个状态。
- Context把行为委托给所组合的当前状态对象。
- 通过把每个状态封装进一个类，我们把以后需要做的任何改变局部化了。
- 状态和策略模式有相同的类图，但它们的意图不同。
- 策略模式通常会用行为或算法来配置Context类。
- 状态模式允许Context随着状态的改变而改变行为。
- 状态迁移可以由State类或Context类控制。
- 使用状态模式通常会导致设计中类数目增加。
- 状态类可以在多个Context实例之间共享。



设计谜题解答

为糖果机画一张状态图，处理10中1的竞赛。在这个竞赛中，有10%的机率“已售出”状态会导致掉下两颗糖果，而不是一颗。答案如下。





削尖你的铅笔 答案

以下哪些选项描述了我们实现的状态（多选）？答案如下。

- ☒ A. 这份代码肯定没有遵守开放-关闭原则。
- ☒ B. 这份代码会让FORTRAN程序员感到骄傲。
- ☒ C. 这个设计不是很面向对象。
- ☒ D. 状态迁移不明显，被埋藏在一堆条件语句中。
- ☒ E. 我们没有封装变化的部分。
- ☒ F. 未来添加的代码有可能导致bug。



削尖你的铅笔 答案

还剩下一个类没有实现：SoldOutState。为什么你不实现它呢？小心而全面地思考，在每种情况下，糖果机应该有什么样的行为。答案如下。

```
public class SoldOutState implements State {
    GumballMachine gumballMachine;

    public SoldOutState(GumballMachine gumballMachine) {
        this.gumballMachine = gumballMachine;
    }

    public void insertQuarter() {
        System.out.println("You can't insert a quarter, the machine is sold out");
    }

    public void ejectQuarter() {
        System.out.println("You can't eject, you haven't inserted a quarter yet");
    }

    public void turnCrank() {
        System.out.println("You turned, but there are no gumballs");
    }

    public void dispense() {
        System.out.println("No gumball dispensed");
    }
}
```

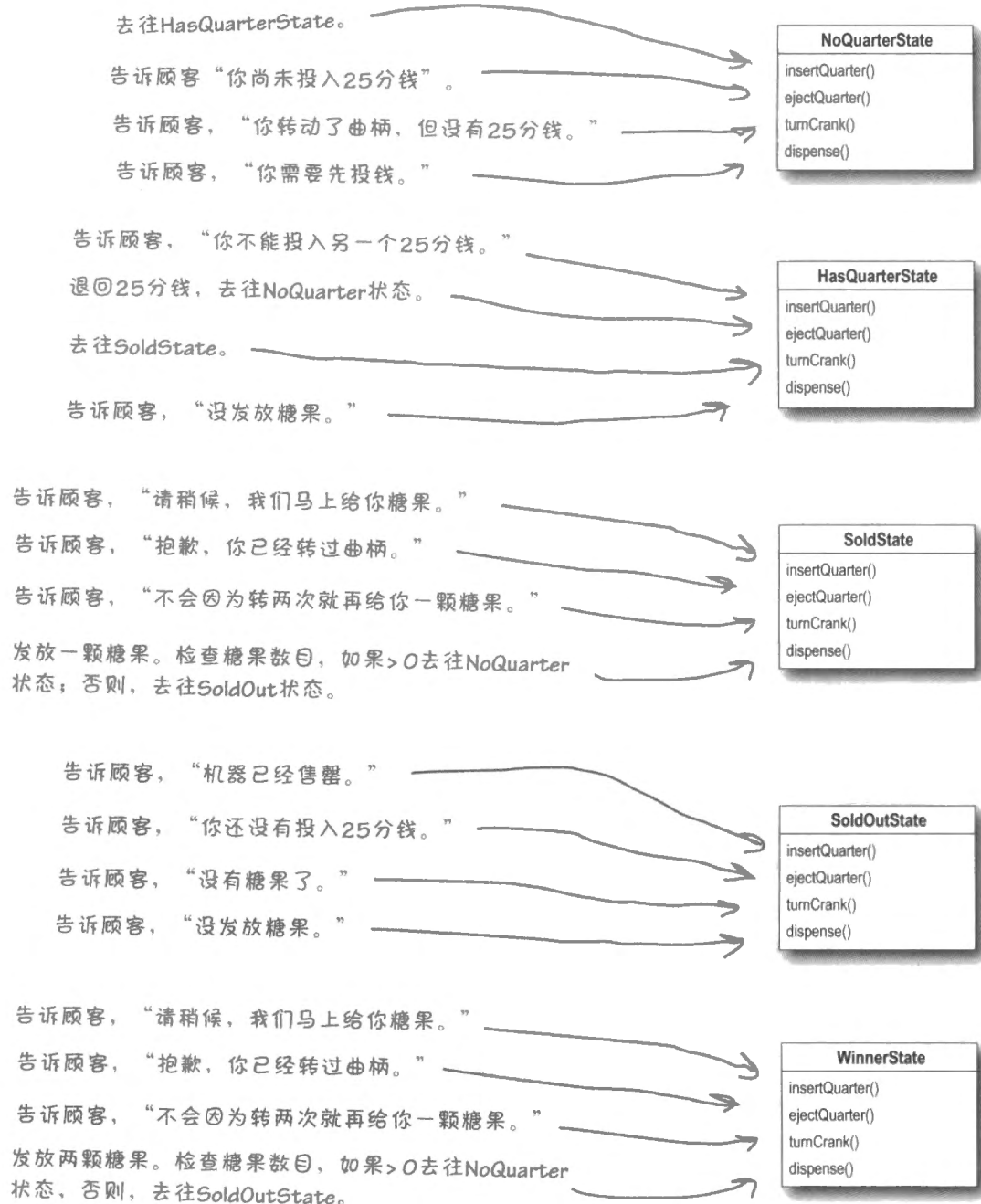
在售罄状态，除非有人重新充填糖果机，否则我们不能做任何事情。



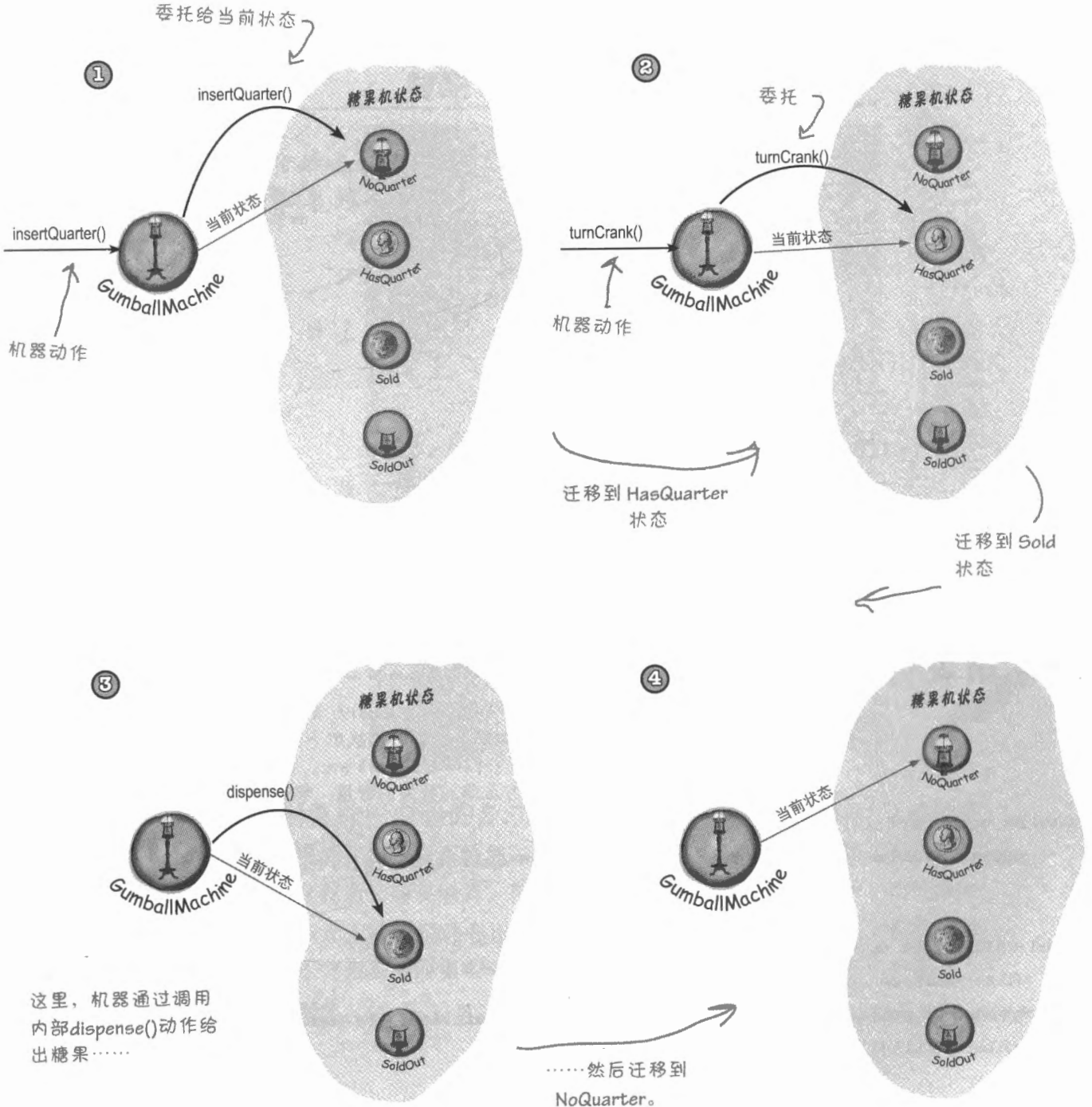
削尖你的铅笔

答案

要实现我们的状态，我们首先需要指定当每个动作被调用时类的行为。请在下图为每个类的每个动作的行为加上注释，答案如下。



幕后花絮： 自助游方案



WHO DOES WHAT?

答案

请将下列模式和描述配对：

模式

描述

状态

封装可以互换的行为，并使用委托来决定使用哪个行为。

策略

子类决定怎样实现算法的步骤。

模板方法

封装基于状态的行为并把行为委托给当前状态。



削尖你的铅笔

答案

要重新充填糖果机，我们添加一个refill()方法到State接口，每个状态必须实现它。除了SoldOutState之外的每一个状态下，该方法不做任何事情。在SoldOutState，refill()迁移到NoQuarterState。我们也添加一个refill()方法到GumballMachine，该方法添加糖果的数量，然后调用当前状态的refill()方法。

```
public void refill() {
    gumballMachine.setState(gumballMachine.getNoQuarterState());
}
```

← 我们添加这个方法到
SoldOutState……

```
void refill(int count) {
    this.count += count;
    System.out.println("The gumball machine was just refilled; its new count is: " + this.count);
    state.refill();
}
```

← ……并添加这个方法到
GumballMachine类。